MACHINE LEARNING REVISION GUIDE

A Practical, Formula-Driven Reference on ML Paradigms, Evaluation Metrics, Ensembles & Pandas Wrangling

A Concise, Visual, and Mechanism-Focused Interview & Revision Handbook

METADATA FIELD	SPECIFICATION
Subject:	Applied Machine Learning Foundations
Target Audience:	SDE / Technical Interviews (Top Tech, FAANG, Product Firms)
Methodology:	Internal Mechanics • State Transitions • Verified Code Recipes • Trade-offs
Source Files Consolidated:	ml/ml_notes.md, ml/ml_core_practice.py, ml/practise.ipynb
Subject Coverage:	Supervised/Unsupervised/Semi-Supervised, Why Models Fail, Evaluation Metrics, Tree Ensembles, XGBoost, Pandas
Key Architectural Focus:	Confusion Matrix Formulas, Model Selection Decision Trees, Bagging vs Boosting & Cohort Analysis
Target Evaluation:	Technical Interview Placement Preparation - Core Engineering Track

CH	CHAPTER TITLE & CORE TOPIC	CRITICAL INTERVIEW CONCEPTS & FORMULAS
1	Machine Learning Paradigms & Archetypes	Supervised (Classification vs Regression), Unsupervised, Semi-Supervised (Medical Imaging)
2	Why Models Fail: Machine Learning Failure Modes	Overfitting vs Underfitting, Concept Drift, Data Leakage, Insufficient Data, Outliers
3	Model Evaluation Metrics & Confusion Matrix	TP/FP/FN/TN, Accuracy, Precision, Recall/Sensitivity, Specificity, F1 Harmonic Mean, ROC-AUC
4	Scikit-Learn Estimator Pipeline & Workflow	Estimator API, `fit()`, `predict()`, `train_test_split()`, Data matrix X and label vector y
5	Classification Models: Logistic Regression & Decision Trees	Sigmoid vs Softmax, Linear boundary limitations, Decision Trees max_depth & instability
6	Ensemble Learning: Random Forests vs Gradient Boosting	Bagging (Parallel Independent Trees Voting) vs Boosting (Sequential Residual Correction, XGBoost)
7	Data Wrangling with Pandas Essentials	DataFrames, `read_csv`, `describe`, `isnull().sum()`, `iloc`, `dropna(thresh)`, `fillna(mean)`
8	Cohort Analysis & User Retention Interpretation	Cohort definition, Time-series retention tables, Churn and behavioral trends

CHAPTER 1 - MACHINE LEARNING PARADIGMS & ARCHETYPES

Machine Learning algorithms discover statistical relationships from data without explicit rule programming. The three primary paradigms differ in their reliance on labeled ground-truth supervision.

Paradigm	Input Data Requirement	Core Sub-Categories	Real-World Application
Supervised Learning	Fully Labeled Dataset (X, y).	**Classification:** Discrete categorical classes. **Regression:** Continuous real-valued outputs.	Spam filtering, medical diagnosis, housing price prediction.
Unsupervised Learning	Unlabeled Dataset (X) only.	**Clustering:** K-Means, DBSCAN. **Dimensionality Reduction:** PCA, Autoencoders. **Association:** Apriori.	Customer segmentation, noise removal, market basket analysis.
Semi-Supervised Learning	Small labeled subset + massive unlabeled dataset.	Pseudo-labeling, transductive learning, consistency regularization.	Medical imaging: 1,000 labeled radiology scans + 1,000,000 unlabeled X-rays.

1.2 Classification vs Regression

• **Classification:** Predicts discrete categories. Uses the **Sigmoid activation function** (1 / (1 + e^-z)) for binary outcomes (0 or 1), and the **Softmax function** for multi-class probability distributions.
• **Regression:** Predicts continuous unbounded quantities (prices, temperatures). Uses linear output activations evaluated via Mean Squared Error (MSE) or Mean Absolute Error (MAE).

CHAPTER 2 - WHY MODELS FAIL: MACHINE LEARNING FAILURE MODES

Failure Mode	Technical Definition	Symptom / Diagnostic Indicator	Remedy / Countermeasure
Overfitting	Model memorizes training noise and high-variance details.	High training accuracy (>99%), but poor testing/validation accuracy.	Regularization (L1/L2), pruning decision trees, dropout, adding training data.
Underfitting	Model is excessively simple (high bias) to capture the underlying pattern.	Low accuracy on BOTH training and testing datasets.	Increase model capacity (add polynomial features, deeper trees, non-linear kernels).
Concept Drift	Statistical relationship between features and target changes over time.	Model performs well initially in production, but accuracy degrades over months.	Continuous retraining pipelines, monitoring distribution drift (KS test).
Data Leakage	Information from the test set or future state leaks into training data.	Unrealistically high training and test scores; fails completely in production.	Split data BEFORE feature scaling/preprocessing; prevent look-ahead bias.
Outliers	Extreme anomaly data points distort statistical models.	Disproportionately drags mean, regression slopes, and decision boundaries.	Trimming, IQR filtering (1.5 * IQR rule), or robust loss functions (Huber loss).

CHAPTER 3 - MODEL EVALUATION METRICS & CONFUSION MATRIX

3.1 The 2x2 Binary Confusion Matrix

Reality \ Prediction	Predicted Positive (1)	Predicted Negative (0)
Actual Positive (1)	**True Positive (TP)**: Correctly caught positive.	**False Negative (FN)**: Missed positive (Type II Error).
Actual Negative (0)	**False Positive (FP)**: False alarm (Type I Error).	**True Negative (TN)**: Correctly cleared negative.

3.2 Core Evaluation Metric Formulas

Metric Name	Mathematical Formula	Intuitive Mnemonic	Crucial Interview Use Case
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall percentage of correct predictions.	Balanced datasets ONLY. Horrible for fraud (99.9% legit = fake 99.9% accuracy!).
Precision	TP / (TP + FP)	*'No innocent person should be punished.'*	Spam filtering, YouTube recommendations (avoid annoying false positives).
Recall (Sensitivity)	TP / (TP + FN)	*'Did I catch all the bad guys?'*	Cancer screening, credit card fraud detection (minimize False Negatives!).
F1-Score	2 * (Precision * Recall) / (Precision + Recall)	Harmonic mean of Precision and Recall.	Balances precision and recall; drops sharply if either metric collapses.
Specificity	TN / (TN + FP)	True Negative Rate (opposite of Recall).	Medical tests: correctly identifying healthy patients as negative.

3.3 ROC Curve & AUC Single-Number Summary

• **ROC Curve (Receiver Operating Characteristic):** Plots True Positive Rate (Recall) vs False Positive Rate (1 - Specificity) as the classification probability threshold varies from 0.0 to 1.0.
• **AUC (Area Under the Curve):** Single-number measure of ranking capability:
- AUC = 1.0: Perfect classifier.
- AUC = 0.9: Excellent discriminative power.
- AUC = 0.5: Worthless (equivalent to random coin toss).
- AUC < 0.5: Worse than random (simply invert the predictions to get >0.5!).

CHAPTER 4 - SCIKIT-LEARN ESTIMATOR PIPELINE & WORKFLOW

Scikit-learn standardizes all machine learning algorithms into a uniform, composable object-oriented interface. Every model is an **Estimator** operating on a 2D feature matrix X and a 1D target vector y.

Code: Complete Scikit-Learn Training Pipeline

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.datasets import load_iris

# 1. Load Data
X, y = load_iris(return_X_y=True)

# 2. Split into Train & Test (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Instantiate Estimator & Fit (Train)
model = LogisticRegression(multi_class="multinomial", max_iter=1000)
model.fit(X_train, y_train)

# 4. Predict & Evaluate
y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

CHAPTER 5 - LOGISTIC REGRESSION & DECISION TREES

Model	Underlying Mathematics	Decision Boundary	Pros & Vulnerabilities
Logistic Regression	Applies Sigmoid function to linear combination: P(y=1) = 1 / (1 + e^-(w^T x + b)).	Strictly **Linear** hyperplane.	Fast, explainable coefficients. **Fails on non-linear datasets** (e.g., XOR pattern).
Decision Tree	Recursively splits feature space to maximize **Information Gain** (or minimize Gini Impurity).	**Orthogonal step-wise (axis-aligned)** boundaries.	Captures non-linear relationships; highly interpretable. **High variance:** small data changes produce completely different trees; overfits easily.

5.2 When to Choose Which Model (Decision Tree)

• **Output is a continuous number?** -> Use Regression (Linear Regression, Ridge/Lasso, Gradient Boosted Regressor).
• **Output is a discrete category?** -> Classification:
- Need simple, explainable baseline? -> **Logistic Regression**.
- Need human-readable if-then decision rules? -> **Decision Tree**.
- Need high accuracy without hyperparameter tuning hell? -> **Random Forest**.
- Tabular competition data where 0.1% accuracy matters? -> **XGBoost / LightGBM**.

CHAPTER 6 - ENSEMBLE LEARNING: RANDOM FORESTS VS GRADIENT BOOSTING

Dimension	Random Forests (Bagging)	Gradient Boosting / XGBoost (Boosting)
Tree Architecture	Parallel, fully independent trees.	**Sequential dependency:** each tree learns from previous mistakes.
Ensemble Technique	**Bootstrap Aggregation (Bagging):** random row samples + random feature subsets per split.	**Gradient Descent on Residuals:** fits each new tree to the negative gradient (errors) of prior predictions.
Aggregation Method	Majority Voting (Classification) or Average (Regression).	Weighted additive sum scaled by **Learning Rate (Shrinkage)**.
Variance vs Bias	Primarily **reduces Variance** (prevents overfitting).	Primarily **reduces Bias** (creates highly capable complex boundaries).
Hyperparameters	`n_estimators` (100-500), `max_features`.	`n_estimators`, `learning_rate` (0.01 - 0.1), `max_depth` (3-6).

Code: XGBoost Production Boilerplate

```
# XGBoost Classifier Configuration (from ml_core_practice.py)
from xgboost import XGBClassifier

model = XGBClassifier(
    n_estimators=100,     # Number of sequential boosting trees
    learning_rate=0.1,    # Shrinkage factor (scales contribution of each tree)
    max_depth=3,          # Shallow trees prevent overfitting
    random_state=42
)
model.fit(X_train, y_train)
```

CHAPTER 7 - DATA WRANGLING WITH PANDAS ESSENTIALS

7.1 Master Pandas Data Cleaning Functions (from ml_core_practice.py)

Code: Pandas Core Wrangling Recipes

```
import pandas as pd

# 1. Loading & Overview
df = pd.read_csv("dataset.csv")
print(df.head(5))         # Inspect first 5 rows
print(df.describe())       # Summary statistics: count, mean, std, min, 25%, 50%, max
print(df.shape)            # (row_count, col_count)

# 2. Missing Value Imputation & Cleaning
missing_counts = df.isnull().sum()  # Count nulls per column
threshold = len(df) * 0.5           # 50% threshold
df = df.dropna(axis=1, thresh=threshold)  # Drop columns with >50% nulls
df['Age'] = df['Age'].fillna(df['Age'].mean())  # Impute missing values with mean

# 3. Filtering & Indexing
adults = df[df['Age'] >= 18]         # Boolean mask filtering
sample_rows = df.iloc[1:4, 0:2]      # Integer-based indexing: rows 1-3, cols 0-1

# 4. Multi-Column Sorting
df_sorted = df.sort_values(by=["Price", "Qty"], ascending=[False, True])
```

CHAPTER 8 - COHORT ANALYSIS & USER RETENTION INTERPRETATION

A **Cohort** is a group of users who experienced a common event (e.g., signup, purchase) during a shared time interval. Cohort analysis tracks behavioral retention and churn degradation over subsequent time periods.

Signup Cohort	Users Joined	Month 1 Active	Month 2 Active	Month 3 Active	Retention Insight
January	100	80 (80%)	60 (60%)	50 (50%)	Stable 50% long-term baseline retention.
February	200	180 (90%)	150 (75%)	120 (60%)	Higher initial retention (product feature improvement).
March	300	210 (70%)	120 (40%)	90 (30%)	Steep churn drop-off (potential buggy release or lower quality acquisition channel).

MACHINE LEARNING INTERVIEW CHEAT-SHEET

- **Evaluation Metrics:** Precision = minimize False Positives; Recall = minimize False Negatives; F1 = harmonic mean balance.
- **Why Models Fail:** Overfitting (high variance), Underfitting (high bias), Concept Drift (temporal change), Data Leakage (future info).
- **Random Forest:** Bagging (parallel independent trees) -> reduces variance.
- **Gradient Boosting:** Boosting (sequential residual errors) -> reduces bias.
- **Pandas:** `isnull().sum()`, `dropna(thresh=...)`, `fillna(mean)`, `iloc` integer indexing.